

# Validator-Guided Repair on a Shared Initial LLM Candidate: A Preregistered Paired Evaluation for Network Configuration Safety

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We preregistered and executed a paired confirmatory experiment (PREREG-CAND-001-VER-GVR-2026-09-01) to measure whether permitting a deterministic network-configuration validator plus one automated repair changes the rate of validator-valid executable configurations relative to leaving the identical sampled LLM candidate unguarded (`shared_initial_candidate` semantics). Using the declared generator `gpt-5-mini` (hosted adapter `openai_responses`) and deterministic `netops_validator_v5` on  $N = 200$  paired intents (`completed_episode_count = 400`; `model_calls_used = 200`), every shared initial candidate passed validator checks and no repairs were attempted or applied (`repair_attempt_count = 0`; `repair_applied_count = 0`). Observed outcomes: `baseline_success = 200/200`, `guarded_success = 200/200`; paired contingency  $n11 = 200$ ,  $n10 = 0$ ,  $n01 = 0$ ,  $n00 = 0$ ; exact McNemar two-sided  $p = 1.0$ ; paired bootstrap percentile 95% CI (seed = 1729; 10,000 resamples) = [0.0, 0.0]. We present artifact-grounded methodology, execution accounting, representative validator diagnostics, compact contingency and repair summaries, and a focused discussion clarifying the causal limits of the executed estimand under `shared_initial_candidate` semantics and preregistered follow-on designs required to measure repair efficacy under independent sampling, higher difficulty, or adversarial inputs.

## I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate operator intent into device configuration sequences in network operations (NetOps). Erroneous sequences, syntactic mistakes, or fabricated fields can cause service disruption or policy violations when applied to devices. A common architectural mitigation is a generate→validate→repair (GVR) pipeline: generate candidate configuration(s), deterministically validate them against intent/topology/workflow constraints, and trigger automated repair when the validator finds faults. Prior work documents verifier-guided improvements in infrastructure-as-code and related code-generation domains and recommends grounding strategies to reduce hallucination in operational tasks [1–3]. However, whether verifier-guided pipelines reduce execution-level operational risk for network configurations remains an open empirical question because prior demonstrations often measure textual correctness or IaC test suites rather than executed device-state safety in packet-level or emulator contexts [3,4].

This paper reports a fully preregistered paired confirmatory experiment that isolates the downstream causal contribution of a deterministic validator and any permitted repair acting on the

exact same sampled LLM candidate (`shared_initial_candidate` semantics). Under these semantics baseline and guarded episodes for each intent begin from the identical initial generation; any paired difference can therefore only arise from deterministic validation and a permitted repair of that same candidate. Our primary research question is: under `shared_initial_candidate` semantics, does permitting a deterministic validator plus at most one automated repair change the proportion of validator-valid executable configurations relative to leaving the same initial candidate unguarded? The contribution is an artifact-grounded report of the executed estimand with complete execution accounting, exact inferential statistics, representative validator diagnostics, and precise recommendations for preregistered follow-on designs that can measure repair efficacy when independent sampling, elevated difficulty, or adversarial inputs are employed.

## II. RELATED WORK

Hallucination and factual unreliability in LLM outputs are well documented and motivate grounding, retrieval, and verification strategies for operational tasks [1]. Retrieval-augmented and evidence-grounded pipelines can reduce hallucination in operations-style QA, though these works evaluate textual or documentation correctness rather than device-level executability [2]. Generate→verify→repair architectures have shown empirical improvements in IaC and program generation (for example, verifier-guided fine-tuning and policy-guided repair loops) and are a primary methodological inspiration for our work [3]. Recent NetOps evaluations examine intent translation and LLM capability but emphasize capability benchmarks rather than standardized execution-level safety measures that map generated commands to executable device states in a simulator/emulator [4]. Comprehensive AIOps surveys highlight the need for execution-aware benchmarks and validator fidelity uplift to bridge generation to operational safety [5].

From these verified sources we identified two methodological gaps that motivated the current paired design. First, verifier-guided demonstrations in adjacent domains do not directly measure whether corrections prevent unsafe executable states when enacted on networks; empirical transferability to NetOps therefore requires execution-level validation. Second, many prior studies mix sampling variability with downstream validator/repair effects; precisely defined estimands that separate these sources of variation are uncommon. We therefore

preregistered a shared\_initial\_candidate paired design that isolates the downstream validator/repair effect on identical generator outputs.

### III. METHODOLOGY

Preregistration, estimand, and paired semantics. We preregistered the study as PREREG-CAND-001-VER-GVR-2026-09-01 and froze the primary estimand before any model calls. Primary estimand: paired\_success\_rate\_difference\_guarded\_minus\_baseline (the per-intent paired difference in the proportion of validator-valid executable configurations). The design is a confirmatory paired binary comparison over task\_count = 200 intents producing planned\_episode\_count = 400 episodes. Crucially, generation semantics were registered and executed as shared\_initial\_candidate: exactly one sampled initial generation per pair was produced and deterministically reused by both baseline and guarded episodes. This design choice isolates any downstream causal effect to deterministic validation and the permitted repair acting on the identical candidate; it intentionally removes sampling variability between conditions. We emphasize that the archived model\_configuration.json records temperature = null/unset; null/unset is not evidence of deterministic sampling and does not imply temperature = 0.

Generation and provider audit. The declared generator used in execution was gpt-5-mini via the hosted adapter openai\_responses. The execution manifest records model\_calls\_used = 200 (one shared initial generation per pair), hosted\_model\_call\_event\_count = 200, completed\_hosted\_model\_call\_event\_count = 200, failed\_hosted\_model\_call\_event\_count = 0, cache\_miss\_count = 200, and stage\_counts indicating shared\_initial\_generation = 200. These provider-audit counts confirm that the adapter produced a single sampled candidate per pair for reuse by both arms as preregistered. The experiment permitted no retrieval augmentation (RAG=false in the preregistration) and the adapter enforced the maximum\_model\_calls limit in the preregistration.

Task generation, contamination controls, and task manifest. A deterministic task generator produced the 200 synthetic intent\_configuration\_repair\_v1 tasks. Each manifest entry includes initial\_state, expected\_final\_state, required\_sequence, difficulty annotations (e.g., level tags: very\_hard, extreme), allowed\_touched\_fields, and a generation\_seed to permit exact replay. Preexecution contamination controls enforced an exact-match SHA-256 policy and high-similarity n-gram checks (n-gram Jaccard threshold, preregistered high-similarity flag threshold = 0.9). No exact-match contamination exclusions were flagged and no confirmatory exclusions were required; any flagged items would have been moved to an exploratory leaked bucket per the contamination\_plan. Contamination artifacts and the fixed task manifest are archived for audit and replay.

Deterministic validator, repair policy, and scored outcome mapping. Scoring used deterministic\_netops\_validator\_v5

with scoring\_rule = "full\_intent\_constraint\_validation." The validator is deterministic and structured: it parses candidate command sequences into normalized\_configuration, compares parsed\_command\_count to task required\_command\_count, enforces workflow\_policies (final\_group\_constraints, minimum\_active\_in\_groups, must\_be\_down\_for\_fields, protected\_interfaces), evaluates dependency rules, and emits structured validator\_trace objects including validation\_before and validation\_after subtables, violation\_codes, violation\_count, parsed\_command\_count, required\_command\_count, observed\_touched\_field\_count, and a repair\_applied boolean. For guarded episodes a single automated repair attempt was permitted (maximum\_repair\_calls\_per\_task = 1); the repair module, if invoked, would generate an edited candidate to be revalidated. Under shared\_initial\_candidate semantics the baseline arm deterministically reused the same sampled candidate without repair; only the guarded arm could alter that identical candidate by invoking the permitted repair. For analysis, validator.valid == true mapped to a binary "executable/valid" outcome per episode; validator.valid == false would trigger repair\_attempt logic in guarded episodes.

Missingness, exclusions, and analysis procedures. The preregistration prescribed the complete\_pair\_primary missingness policy: exclude incomplete pairs from the primary confirmatory McNemar test. A sensitivity analysis (failure-as-zero) was preregistered to treat any missing or failed episode as unsafe. The primary hypothesis used an exact two-sided McNemar test on paired binary outcomes; paired bootstrap percentile CIs were computed (bootstrap\_seed = 1729; bootstrap\_resamples = 10,000) to quantify uncertainty in the paired difference. Exploratory subgroup comparisons by difficulty pattern and failure category were labeled exploratory and would be adjusted using Benjamini-Hochberg FDR = 0.05 when reported. Multiplicity for the main confirmatory test was controlled by design (single prespecified primary test).

Execution reconciliation and reproducibility artifacts. Execution completed without infrastructure failures and matched preregistered accounting: completed\_episode\_count = 400, failed\_episode\_count = 0, complete\_pairs = 200. Deterministic reconciliation checks verified pair accounting, marginal consistency, and provider audit stages. Per-episode scoring JSON artifacts record the shared\_initial\_candidate, raw\_response\_sha256, normalized\_response, the validator\_trace object, repair\_applied flags, and the scorer\_id/version. Representative artifact excerpts (archived) demonstrate the validator fields used to compute binary outcomes (e.g., parsed\_command\_count, required\_command\_count, normalized\_configuration, violation\_count). All artifacts required to replay the executed estimand (task manifest, responses, scoring traces, analysis CSVs, and model configuration) are archived in the evidence bundle to permit deterministic replay of the exact shared\_initial\_candidate sampling and subsequent scoring.

Implementation notes and limitations of methods. The validator performs static, deterministic checks against task

workflow\_policies and expected command sequences; it does not perform vendor CLI execution nor packet-level emulation. This design choice was deliberate to isolate syntactic and workflow-policy correctness; it is a recognized construct validity limitation for dynamic runtime behaviors. The preregistered repair policy limited automated repairs to one attempt per guarded episode to bound repair budget and analysis interpretation. Finally, the model\_configuration records temperature = null/unset; the methods explicitly note that null/unset is not evidence of deterministic sampling and that sampling nondeterminism may still have occurred during the single sampled generation that was archived and deterministically reused.

#### IV. RESULTS

Execution accounting and deterministic reconciliation. The execution completed exactly as preregistered: planned\_episode\_count = 400, completed\_episode\_count = 400, failed\_episode\_count = 0, and model\_calls\_used = 200 (one shared initial generation per pair). Analysis missingness\_summary reports complete\_pairs = 200 with zero failed or unscorable episodes. Deterministic reconciliation records hosted\_model\_call\_event\_count = 200, completed\_hosted\_model\_call\_event\_count = 200, cache\_miss\_count = 200, and stage\_counts = {"shared\_initial\_generation": 200}; cross\_condition\_cache\_key\_reuse\_observed = false and all deterministic consistency checks passed.

Primary binary outcomes and contingency. The validator mapped each episode to a binary executable outcome (validator.valid). Condition summaries show baseline\_successes = 200/200 and guarded\_successes = 200/200 (success\_rate = 1.0 for both arms). The paired 2×2 contingency table is n11 = 200, n10 = 0, n01 = 0, n00 = 0; exact McNemar two-sided p = 1.0. The paired bootstrap (seed = 1729; 10,000 resamples) produced a percentile 95% CI for the paired difference of [0.0, 0.0]. These files are archived (analysis/condition\_summary.csv and analysis/paired\_contingency\_table.csv) and exactly reproduce the contingency and CI numbers reported here.

Repair activity, validator diagnostics, and representative examples. Across the confirmatory sample the validator flagged no violations on any shared initial candidate; as a result repair\_attempt\_count = 0 and repair\_applied\_count = 0. Representative per-episode scoring JSON artifacts illustrate the pattern. For example, task-000004 (pair\_id pair-000004) shared\_initial\_candidate and both condition responses were identical:

```
interface edge2 admin down interface edge2 vlan 40 interface edge2 admin up interface edge1 admin down
```

For task-000004 the archived validator trace shows parsed\_command\_count = 4, required\_command\_count = 4, observed\_touched\_field\_count = 3, and validation\_after.valid = true. Similarly, task-000005 shows parsed\_command\_count = 3 and required\_command\_count = 3 with validation\_after.valid = true; task-000006 shows parsed\_command\_count = 6 and required\_command\_count =

6 with validation\_after.valid = true. These per-episode diagnostics (execution/scoring/task-00000X-\*.json) demonstrate that the validator parsed the expected command set, matched required\_sequence checks, and reported no violation codes for the sampled candidates. Because the validator never produced a violation, the guarded repair path was never invoked and could not alter any outcome.

Contamination and provider audit summary. Preexecution contamination checks found no exact-match contamination exclusions; analysis/contamination\_summary.csv is empty. Provider audit and execution manifests corroborate the sampling semantics: hosted\_model\_call\_event\_count = 200, cache\_miss\_count = 200, and stage\_counts indicating exactly one shared\_initial\_generation per pair. No infrastructure retries or failures occurred; analysis/analysis\_log.jsonl documents the paired analysis completion and bootstrap parameters (bootstrap\_resamples = 10000; bootstrap\_seed = 1729).

Compact repair summary (explicit). In concise preregistered terms: total\_pairs = 200; repair\_attempt\_count = 0; repair\_applied\_count = 0. The empty repair counts fully explain the degenerate paired contingency (all pairs valid before any repair opportunity). All artifacts supporting these counts are archived in the evidence bundle (notably the execution/scoring JSONs, execution/task\_manifest.jsonl, analysis/paired\_contingency\_table.csv, and analysis/condition\_summary.csv) and permit independent verification that no repairs were produced or applied in guarded episodes.

#### V. DISCUSSION

Interpreting the executed estimand. The shared\_initial\_candidate semantics were intentionally chosen to isolate the causal effect of deterministic validation and any permitted repair acting on the exact same generator output. The executed estimand therefore answers a narrowly defined causal question: when baseline and guarded episodes begin from the identical sampled candidate, can deterministic validation and at most one automated repair change the validator-valid/executable outcome? The observed null effect (paired difference = 0.0) shows that, for this task bank and this single sampled candidate per pair, the validator found no violations and the permitted repair path was never exercised. Importantly, this does not evaluate whether guarded pipelines reduce first-pass generator error rates under independent sampling or different model temperatures; such claims would require a different preregistered estimand and execution.

Concrete validator behavior and why the guarded path was not invoked. The deterministic\_netops\_validator\_v5 used in scoring enforces a structured sequence of checks: lexical/command parsing, mapping commands to normalized\_configuration, counting parsed\_command\_count against required\_command\_count from the task manifest, and evaluating workflow\_policies such as "minimum\_active\_in\_groups" and "must\_be\_down\_for\_fields." The archived per-episode JSON traces show parsed\_command\_count == required\_command\_count

and `validation_after.valid == true` for representative tasks (e.g., task-000004: `parsed_command_count = 4`, `required_command_count = 4`), which means the validator accepted the shared initial candidate as satisfying the syntactic and workflow constraints recorded in the task manifest. Because the validator reported zero `violation_codes` across the confirmatory sample, the guarded branch's single permitted automated repair had no trigger and therefore `repair_attempt_count = 0` and `repair_applied_count = 0`. This artifact-level diagnostic fully explains the degenerate paired contingency ( $n_{11} = 200$ ) observed in the results.

Statistical and power implications given zero discordance. The exact McNemar two-sided  $p = 1.0$  and bootstrap CI =  $[0.0, 0.0]$  are deterministic consequences of  $n_{10} = n_{01} = 0$ . From an inferential perspective, zero discordant pairs carry no information about the sign or magnitude of a potential repair benefit under sampling regimes that would produce validator failures. The preregistered power analysis assumed nonzero baseline unsafe rates (sensitivity checks considered baseline unsafe rates in  $\{0.2, 0.3, 0.4\}$  and target effects  $\approx 0.08\text{--}0.12$ ). Those assumptions remain necessary: to detect an absolute risk reduction on the order of 10 percentage points with  $\sim 80\%$  power requires an observable rate of validator failures under the baseline sampling distribution. In practice this means either (a) selecting task items with a higher expected difficulty/failure label, (b) altering generation sampling to increase first-pass diversity (e.g., nonzero temperature), or (c) injecting adversarial perturbations so that the guarded repair path is exercised. Without such adjustments the paired shared\_initial\_candidate design can produce informative nulls when the validator accepts nearly all sampled candidates, as occurred here.

Construct validity of the validator abstraction. Deterministic\_netops\_validator\_v5 provides strong syntactic and workflow-policy checks and records structured diagnostics that make its operation auditable in artifact traces. However, construct validity limits remain: the validator maps candidate commands to an asserted final\_state and enforces required\_sequence and policy constraints, but it does not execute vendor CLI on hardware nor perform packet-level emulation of run-time protocol interactions. As a result, certain dynamic failure modes (e.g., transient routing convergence, BGP session flaps, or timing-dependent failover behavior) are outside the validator's scope. The evidence bundle therefore supports a qualified conclusion only about validator-level syntactic/workflow correctness for the sampled candidates, not about runtime-observable failures that a live device or packet-level emulator might reveal. Representative task annotations in the manifest (difficulty levels labeled "very\_hard" and an isolated "extreme" instance) indicate the generator was challenged by multi-step required\_sequences, yet the validator still accepted the single sampled generations; this suggests that either the generator produced compliant plans for these synthetic tasks or that the task bank difficulty distribution did not produce the class of subtle errors that would need repair.

Operational implications and conservative interpretation.

Practitioners evaluating GVR pipelines should note two operational lessons from these artifacts. First, an observed `guarded==baseline null` under `shared_initial_candidate` semantics does not imply that a GVR pipeline would be unhelpful under standard deployment conditions (where multiple independent samples, different temperatures, or diverse models are used). Second, artifact diagnostics can reveal the immediate cause of a null: here the cause was absence of validator detections (repair path never invoked). Operational evaluation should therefore pair estimator choices with task selection that yields nontrivial validator-failure rates when the goal is to measure repair efficacy. For safety-critical deployments, a practical path is to strengthen validator fidelity (e.g., integrate vendor CLI semantics, run candidate changes against a packet-level emulator or staged lab, or add dynamic checks for protocol convergence) so that the validator better approximates the operational harm surface that repairs must address.

Methodological recommendations grounded in archived evidence. The execution and manifest artifacts identify specific, artifact-traceable modifications for follow-on preregistered experiments that would place the repair mechanism under test: (1) relax `shared_initial_candidate` semantics to generate independent first-pass candidates per condition so that generator variability can produce baseline failures that repairs might fix (this requires changing `generation_semantics` and re-defining the estimand); (2) increase the expected baseline failure rate via task selection (prefer manifest entries annotated "extreme" or compose adversarial task variants) so that repairs are triggered with nontrivial probability; (3) preregister explicit sampling parameters (temperature sweeps) instead of leaving temperature null/unset; the archived `model_configuration.json` makes clear that `temperature = null` in this run and that null/unset does not imply deterministic sampling; (4) augment the validator by integrating higher-fidelity execution backends (vendor CLI emulation or packet-level simulation) to raise construct validity for runtime failure modes; and (5) increase permitted repair iterations or diversify repair strategies so repairs have more opportunity to converge on a valid candidate when initial violations are detected. Each of these recommendations follows directly from the execution artifacts (e.g., `stage_counts` showing only `shared_initial_generation = 200` and the `repair_applied = false` flags across the sample) and the preregistration sensitivity analysis that assumed nonzero baseline unsafe rates.

Reproducibility and artifact auditing. The artifact bundle contains per-episode validator traces (`validation_before/validation_after`), raw shared initial responses, the execution manifest, `model_configuration.json` (`declared_model_version = "gpt-5-mini"`, `temperature = null`), and `analysis/condition_summary.csv` and `analysis/paired_contingency_table.csv`. These artifacts permit independent auditors to verify that: (a) exactly one shared initial generation per pair was produced and reused in both arms (provider audit: `shared_initial_generation = 200`); (b) `validator.valid` was true for all episodes and no repair prompts or repair traces were created (`repair_attempt_count = 0`);

and (c) the exact contingency counts driving the McNemar test are reproducible. Importantly, auditors should not infer deterministic model sampling from temperature = null; the execution manifest explicitly records that temperature was unset and that sampling nondeterminism cannot be excluded for the single sampled generation per pair.

Threats to external validity and next steps. The principal external validity limits are single-model scope (declared\_model\_version = "gpt-5-mini" via the archived adapter), synthetic deterministic task bank, and the validator abstraction described above. These limits are documented in the registered limitations and in the evidence bundle. The most direct path to broader external validity is a controlled factorial follow-on that crosses (i) independent vs shared generation semantics, (ii) model temperature and model family, and (iii) validator fidelity (syntactic/workflow vs emulator-integrated). Such a design would separate generator error rates from repair efficacy and allow estimation of the number-needed-to-repair under realistic sampling variability. All of these modifications are prespecified in the paper's preregistered follow-on recommendations and can be implemented while preserving the same deterministic task manifest structure for continuity of comparison.

Summary. The expanded artifact diagnostics show that the run produced an informative but narrowly scoped result: deterministic validation found no violations for the single sampled candidate per intent, so the permitted guarded repair path was never exercised and could not change outcomes. This clarifies the causal limit of the executed estimand and points directly to preregistered experimental modifications needed to measure repair efficacy under conditions that produce validator failures and thus allow repairs to act.

## VI. LIMITATIONS

- Shared\_initial\_candidate semantics: the design produced exactly one sampled initial generation per intent and reused it across baseline and guarded conditions; this measures validator/repair effects only and cannot detect differences caused by independent per-condition sampling or prompt engineering.
- Temperature unset (temperature = null) in the archived model configuration: null/unset is not evidence of deterministic sampling and does not imply temperature = 0; sampling nondeterminism may still have occurred during the single sampled generation.
- Single model and single hosted provider: results reflect gpt-5-mini under the archived adapter; generalization to other models or model families is unsupported by these artifacts.
- Synthetic task bank and validator abstraction: tasks were synthetic 'intent\_configuration\_repair\_v1' items and the deterministic validator is an abstraction; realistic vendor CLI idiosyncrasies, emergent runtime interactions, and hardware effects were not executed on live devices.
- No observed validator failures: all initial candidates were validator-valid, so the experiment could not assess the

repair step's effectiveness; the null effect therefore reflects the task bank and sampling semantics rather than evidence that GVR is unnecessary in general.

## VII. CONCLUSION

We executed a fully preregistered paired confirmatory experiment that measures the downstream effect of a deterministic validator plus an allowed single automated repair on the exact same sampled LLM candidate per intent (shared\_initial\_candidate semantics). Using the declared generator gpt-5-mini and deterministic\_netops\_validator\_v5 on 200 paired intents, every shared initial candidate passed validation and no repairs were attempted or applied; guarded and baseline outcomes were identical for the executed estimand (paired difference = 0.0; McNemar  $p = 1.0$ ; paired bootstrap CI = [0.0, 0.0]). This artifact-grounded null result demonstrates an estimand boundary: under shared\_initial\_candidate semantics and the executed task bank the guarded pipeline could not be exercised and therefore could not change outcomes. It does not imply that GVR pipelines are unnecessary generally. Preregistered follow-on experiments that (1) remove shared\_initial\_candidate semantics, (2) increase task difficulty or inject adversarial inputs, (3) set explicit sampling parameters, and (4) raise validator fidelity are required to empirically evaluate repair efficacy under realistic sampling variability and adversarial conditions. The archived artifacts and reconciliation files enable exact reproduction of the executed estimand and provide a secure base for precise preregistered extensions.

References [1] A. Alansari and H. Luqman, "Large Language Models Hallucination: A Comprehensive Survey," arXiv preprint arXiv:2510.06265, 2025. [2] B. Zhang, H. Rao, and D. Zhao, "Evidence-Grounded RAG for Cloud-Native DevOps: Hallucination-Resistant AIOps Question Answering over Private Operations Documents," J. Autom. Cloud-Native Syst., 2024. [3] P. Jana et al., "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback," Proc. ACM Symp. (2026). [4] Y. Miao et al., "An Empirical Study of NetOps Capability of Pre-Trained Large Language Models," arXiv:2309.05557, 2023. [5] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models," ACM Comput. Surv., 2025.

One-line reiteration of the primary estimand limit: the preregistered primary estimand intentionally used shared\_initial\_candidate semantics; evaluating per-condition independent sampling requires a different preregistration and analysis plan (explicitly recommended above).

## DISCLOSURE STATEMENT

Disclosure (concise): The human Research Director selected the broad topic family (Generative AI / LLMs for NetOps) before the autonomy lock. After the autonomy lock the autonomous system performed literature review, experiment selection, preregistration (PREREG-CAND-001-VER-GVR-2026-09-01), code generation, execution, deterministic validation, automated analysis, and manuscript drafting without manual human

editing of technical content. Declared model used in execution: gpt-5-mini via hosted adapter 'openai\_responses' (execution used model\_calls\_used = 200; archived model\_configuration.json records temperature = null/unset; null/unset is not evidence of deterministic sampling and does not imply temperature = 0). Generation semantics executed: shared\_initial\_candidate — exactly one sampled initial generation per intent was produced and deterministically reused for both baseline and guarded conditions. Validator: deterministic\_netops\_validator\_v5 scored candidates using 'full\_intent\_constraint\_validation' and a single automated repair iteration was permitted in guarded episodes; in this run the validator flagged no violations and no repairs were applied (repair\_attempt\_count = 0; repair\_applied\_count = 0). Execution accounting: completed\_episode\_count = 400 (200 paired intents) completed without preregistration deviations. Master prompt immutable reference: provenance/master\_prompt.txt — SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba. Pre-lock infrastructure development and rehearsal occurred but pre-lock artifacts were not used as empirical evidence in this final run. After the autonomy lock no human manually rewrote technical results, manuscript text, figures, or references.

#### REFERENCES

- [1] Aisha Alansari, Hamzah Luqman, "Large Language Models Hallucination: A Comprehensive Survey", 2025, 10.48550/arxiv.2510.06265
- [2] Boning Zhang, Hengning Rao, Derek Zhao, "Evidence-Grounded RAG for Cloud-Native DevOps: Hallucination-Resistant AIOps Question Answering over Private Operations Documents", 2024, 10.69987/jacs.2024.40308
- [3] Prithwish Jana, Sam Davidson, Bhavana Bhasker, Andrey Kan, Anoop Deoras, Laurent Callot, "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback", 2026, 10.1145/3786583.3786898
- [4] Yukai Miao, Yu Bai, Li Chen, Dan Li, Haifeng Sun, Xizheng Wang, Ziqiu Luo, Ren, Yanyu, Dapeng Sun, Xiuting Xu, Qi Zhang, Chao Xiang, Xinchu Li, "An Empirical Study of NetOps Capability of Pre-Trained Large Language Models", 2023, 10.48550/arxiv.2309.05557
- [5] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635